Project Report: Supermarket Checkout System

Object Oriented Software Engineering - CentraleSupélec

Moncef Ait Belkacem
Raïcha Ibrah Sanoussi

May 2026

Contents

1 Introduction

This report presents the implementation of a Supermarket Checkout System developed in Java.

The goal of the project is to provide a software solution for managing supermarket purchases, customers, inventory, and payments through a command-line interface. The system supports different user roles, product categories, discount plans, and stock management operations.

The following sections describe the main design decisions, the architecture of the system, the design patterns employed, and the testing strategy adopted during development.

2 Design Decisions

First, the system is split into different parts, each one handling a specific responsibility. User management, customer management, inventory, discount rules, and the command-line interface are all separated into different classes. This makes the code easier to understand and reduces the risk of breaking other parts when something is changed.

Users are represented with a small class structure. A base User class stores common login information, while Manager and Cashier extend it to represent different roles. This makes it easier to manage permissions in a consistent way.

Customers are handled separately from employees. The Customer class stores personal data and the discount plan, while CustomerBase is used to add and retrieve customers. This keeps all customer logic in one place.

Inventory is managed by a single Inventory class. Each product is represented by an Item, and items are linked to a category using ItemCategory. Categories are handled dynamically through CategoryBase, so new categories can be added without changing existing code.

Custom exceptions are used to handle errors like duplicate users, missing items, or wrong login attempts. This makes errors clearer and easier to debug.

Finally, the command-line interface is kept separate from the business logic. The SuperMarketCheckoutSystem class only reads commands and sends them to the right components. This keeps the system organized and easier to extend.

3 Design Patterns

3.1 Strategy Pattern

The Strategy Pattern is used for customer discount plans.

We define a common interface called `DiscountPlan`, and each discount type implements it. For example, `NormalDiscountPlan`, `PrimeDiscountPlan`, and `PlatinumDiscountPlan` each contain their own way of calculating discounts.

This makes it easy to add new discount plans later without changing the checkout code, which matches requirement R5b. The Strategy Pattern is also indirectly used in the Checkout process, where different customer plans influence delivery pricing and fee computation.

Advantages:

- New discount plans can be added easily.
- The checkout code does not depend on specific discount rules.
- The logic for each plan stays separated and clear.

3.2 Repository Pattern

We also use simple repository-style classes to store and access data.

For example:

- `UserBase` stores employees,
- `CustomerBase` stores customers,
- `CategoryBase` stores item categories.

These classes act as a single place to manage data, and they also help prevent issues like duplicate users or missing entries.

Advantages:

- Data is managed in one place.
- Code is easier to maintain.
- Basic checks (like duplicates) are handled centrally.

3.3 Facade Pattern

The `SuperMarketCheckoutSystem` class works as a kind of facade.

Instead of the user dealing directly with inventory, customers, or payment modules, everything goes through the command-line interface. The system receives commands and forwards them to the right components.

This keeps things simpler for the user and avoids tight links between the interface and the internal logic.

Advantages:

- Easier to use from the outside.
- Less direct dependency between components.
- The internal structure can change without affecting the interface too much.

3.4 Session Management

Authentication is handled by the `UserSession` class.

Instead of keeping login information everywhere in the system, we store the current logged-in user in one place. This class handles login, logout, and also checks what the user is allowed to do.

This keeps authentication separate from the rest of the system and makes it easier to manage user access.

4 System Architecture

The system is organised into several independent components, each responsible for a specific domain of the application.

4.1 Global Structure

The system is organised into several main modules, each responsible for a specific part of the application. This separation makes the code easier to understand, maintain, and extend.

The Command-Line Interface (CLI), implemented in the `SuperMarketCheckoutSystem` class, is the entry point of the system. It reads user input, interprets commands, and sends them to the correct components for execution.

User-related features are grouped in the User Management module, which includes the `User`, `Manager`, and `Cashier` classes, as well as `UserBase` and `UserSession` for handling registration and login.

Customer data is managed separately in the Customer Management module, implemented with the `Customer` and `CustomerBase` classes. This keeps customers and employees clearly separated.

Inventory management is handled by the Inventory module, which uses the `Inventory`, `Item`, and `CategoryBase` classes to manage products and their categories.

The Discount System contains the discount logic. It is based on the `DiscountPlan` interface and its implementations, which makes it easy to add new discount types later.

Finally, the Payment System handles transactions using the `PointOfSale`, `TransactionAuthorisati` and `SampleBankTas` classes to simulate bank payments.

4.2 Core Design Principle

The architecture follows a layered approach.

The presentation layer is represented by the CLI, which receives and processes user input.

The application layer contains the system logic, including command handling, session management, and business workflows.

The domain layer contains the core business entities such as customers, items, and inventory.

This separation ensures that the business logic is not tightly coupled with user interaction.

4.3 Command Processing Flow

User commands are processed in the `SuperMarketCheckoutSystem` class. The workflow is as follows:

1. The user enters a command through the CLI.
2. The command is split into a command name and arguments.
3. The `CommandUtils` class checks whether the command is valid for the current user role.
4. The corresponding handler method is executed.
5. The related business components (inventory, users, customers) are updated.

This design keeps command parsing separate from business logic execution.

4.4 Data Management

The system uses in-memory repositories to store application data. Classes such as `UserBase`, `CustomerBase`, `Inventory`, and `CategoryBase` are responsible for storing and retrieving data.

These components act as simple repositories and help keep data access consistent and well organised.

5 UML Modeling

The UML class diagram of the Supermarket Checkout System shows the main entities of the application and how they are related. It reflects the modular structure of the system and highlights the separation between users, customers, inventory, and discount features.

5.1 Main Classes and Relationships

The system is built around several core parts:

- User hierarchy: The abstract sealed class `User` contains common authentication data. It is extended by `Manager` and `Cashier`, which represent the two types of employees.
- Customer management: The `Customer` class represents a supermarket customer. It stores personal information and a `DiscountPlan`. The `CustomerBase` class is used to store and retrieve customers.
- Inventory system: The `Inventory` class manages items and stock levels. Each `Item` belongs to an `ItemCategory`, and categories are managed dynamically using the `CategoryBase` class.
- Discount system: The `DiscountPlan` interface defines the general behaviour of discount strategies. It is implemented by `NormalDiscountPlan`, `PrimeDiscountPlan`, and `PlatinumDiscountPlan`.
- Session management: The `UserSession` class manages login state and controls access based on user roles.
- System controller: The `SuperMarketCheckoutSystem` class is the main entry point and coordinates all operations.

5.2 Key Design Relationships

The main relationships in the system are:

- Inheritance between `User`, `Manager`, and `Cashier`.
- Composition between `Customer` and `DiscountPlan`.
- Association between `Inventory` and `Item`.
- Association between `Item` and `ItemCategory`.
- Aggregation of repositories inside the main system class.

5.3 UML Diagram

5.4 Sequence Diagram: Payment Process

In addition to the class diagram, a sequence diagram was created to model the payment workflow. It shows how the main system components interact during a payment.

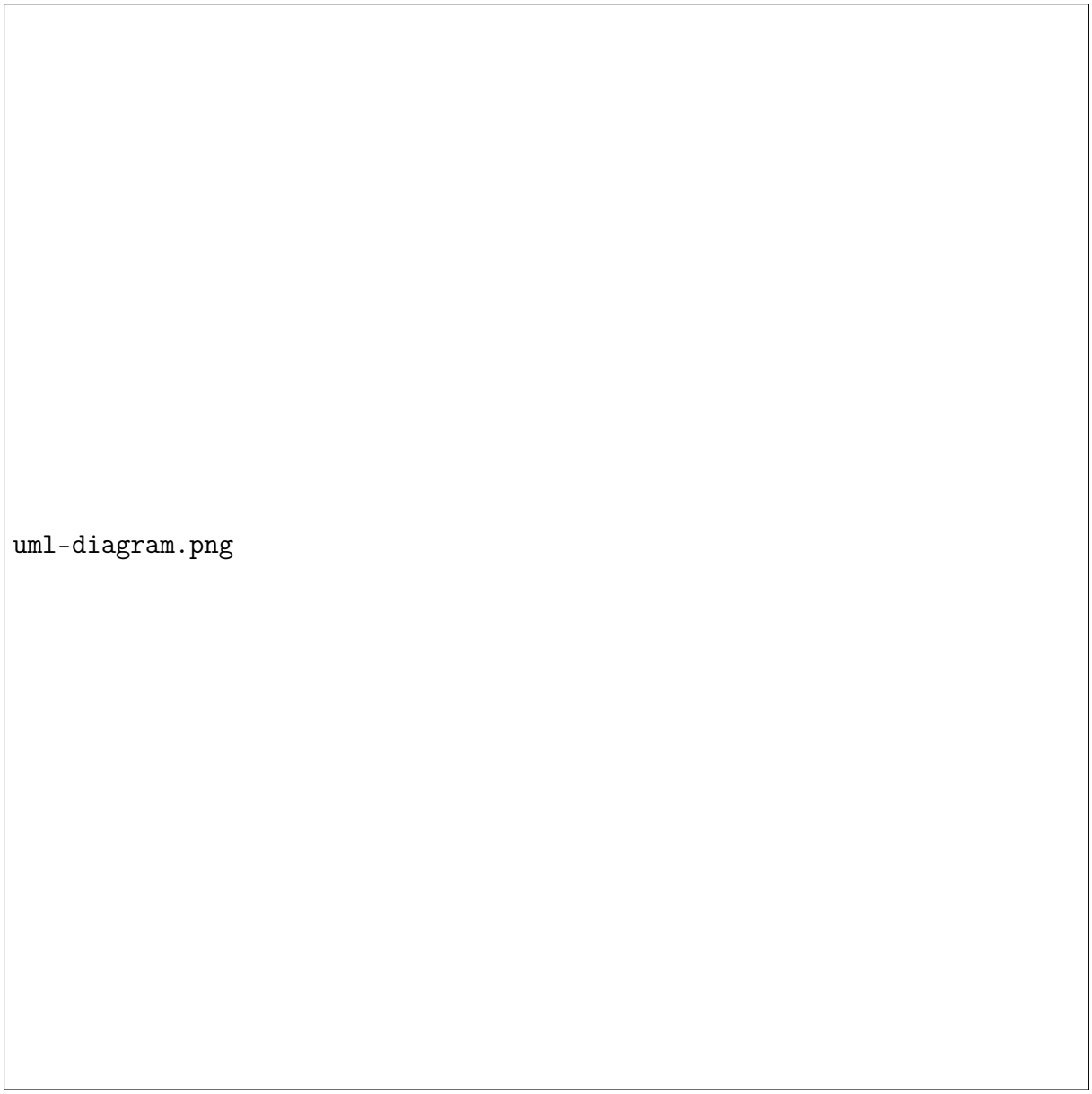

Figure 1: UML diagram

The process starts when a cashier requests a payment through the command-line interface. The system gets the total amount from the current checkout and sends the payment request to the `PointOfSale` component. The POS then communicates with the `TransactionAuthorisationSystem` implementation (`SampleBankTas`) to verify the card number, check the PIN, confirm the account balance, and complete the debit operation.

If the payment is successful, the inventory is updated and the cashier is notified. If it fails, an exception is raised and an error message is displayed.

This sequence diagram complements the class diagram by showing how the main components work together at runtime, including the checkout system, the POS, and the banking authorisation service.

6 Command-Line Interface Design

The system is controlled through a command-line interface (CLI) implemented in the `SuperMarketCheckoutSystem` class. It allows users to interact with the system using text commands.

Each input is read as a single line and split into a command name and a list of arguments. The first part is the command, and the rest are parameters.

This approach keeps the system simple while still supporting many different operations.

Before executing a command, the system checks:

- if the command is valid using the `CommandUtils` class,
- if the user has permission based on their role (Manager, Cashier, or logged out).

This prevents unauthorized actions.
The system uses role-based access control:

- Managers can perform administrative tasks such as user registration and inventory management.
- Cashiers can handle checkout operations such as scanning items and processing payments.
- Some commands are available without login (for example help or runTest).

After validation, commands are sent to specific handler methods in the main system class. Each handler performs a specific task, such as:

- user authentication (login/logout),
- inventory updates (addItem, restock),

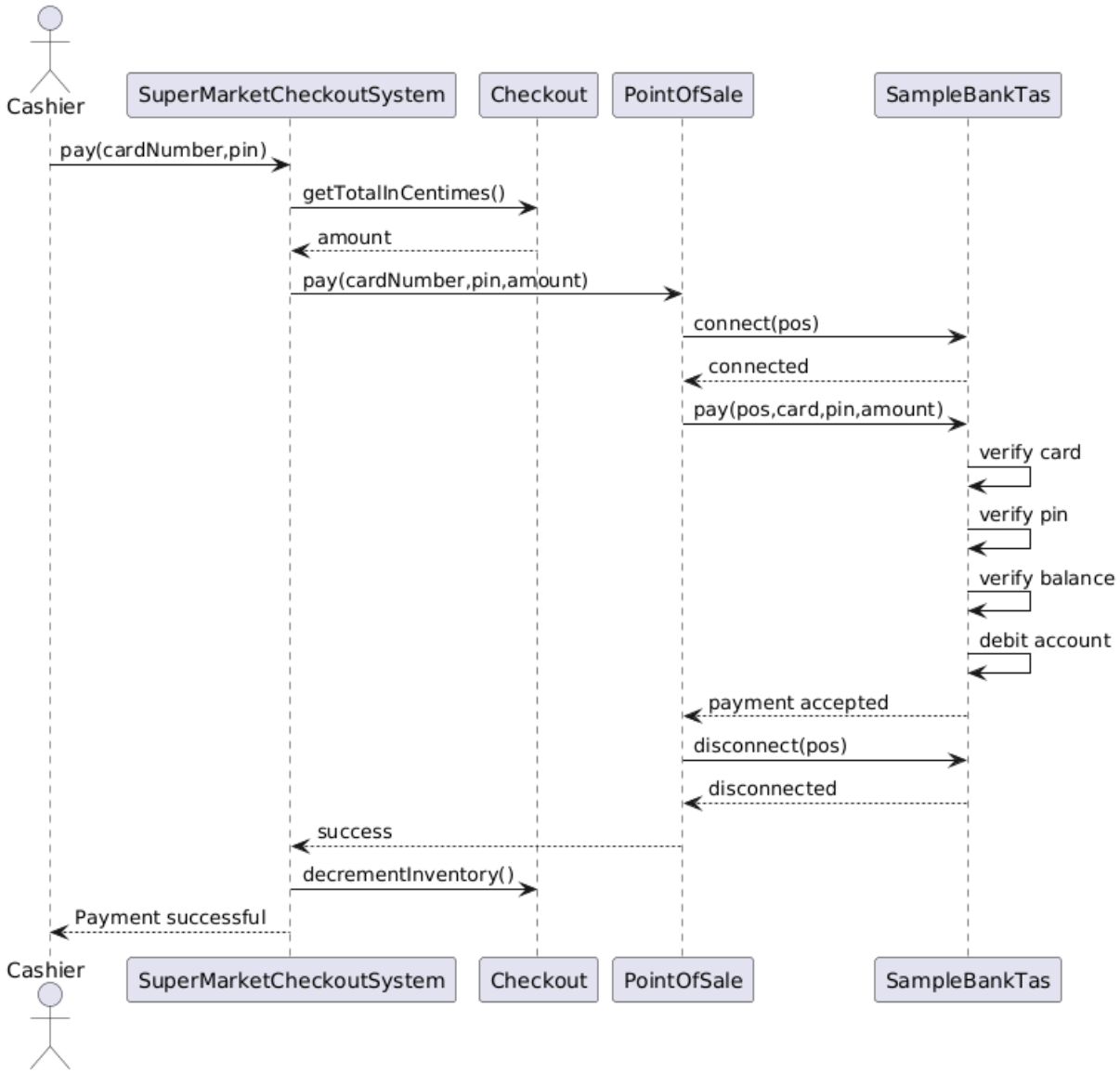

Figure 2: Sequence diagram of the payment process

- customer management (registerCustomer),
- system monitoring (showInventory).

This design separates command parsing from business logic. It makes the system easier to extend, since new commands can be added without changing the core structure.

7 Functional Requirements Implementation (R1-R8)

7.1 R1 - Purchase Management

Purchases are managed through the Checkout class. A checkout session stores scanned items and their quantities, calculates the total bill, applies category discounts and customer discount plans, and handles payment processing.

This design allows the system to fully build a purchase before final checkout and payment.

7.2 R2 - Item Management

Each purchase is made of items represented by the `Item` class. An item is defined by its name, category, unit price, and weight.

Items must be registered in the `Inventory` before they can be used, which ensures consistency between sales and stock.

7.3 R2b - Product Categories

Items are organised into categories such as fruits and vegetables, dairy, and meat. This is implemented using the `ItemCategory` class.

The `CategoryBase` manages all categories and allows new ones to be added dynamically when needed, making the system more flexible.

7.4 R3 - Pricing of Items

Each item has a unit price in centimes to avoid floating-point errors. The total price is calculated based on the quantity of items in the cart.

This ensures accurate and reliable financial calculations.

7.5 R4 - Payment System (POS Design)

Payments are handled through a Point-of-Sale (POS) component represented by the `PointOfSale` class.

To separate the checkout system from banking logic, the payment process uses the `TransactionAuthorisationSystem` interface.

The current implementation uses `SampleBankTas`, which simulates a banking system with card registration, balance checking, and PIN verification.

The payment process works as follows:

1. The POS connects to the authorisation system.
2. The card number and PIN are verified.
3. The account balance is checked.
4. The payment amount is debited.
5. The POS disconnects from the authorisation system.

The Checkout component also handles delivery cost computation based on the total weight of items and the delivery distance. It interacts with an AddressBookService to obtain distances and with a DeliveryHub to manage delivery scheduling. Specific exceptions are used to handle invalid cards, wrong PINs, insufficient balance, and connection errors.

7.6 R5 - Customer Discount Plans

Customers can subscribe to different discount plans using the `DiscountPlan` interface. The system currently supports:

- Normal: no discount,
- Prime: conditional discount based on the total amount,
- Platinum: fixed discount on all purchases.

Each plan defines its own discount logic, which is applied during checkout.

7.7 R5b - Extensible Discount System

The discount system is based on the Strategy pattern, which allows new discount plans to be added without changing the checkout logic. This improves flexibility and scalability.

7.8 R6 - Category-Based Pricing Policies

The system supports pricing rules at the category level. Each item belongs to a category, which can be used to apply specific pricing adjustments or discounts.

This allows different product groups to follow different pricing strategies.

7.9 R6b - Dynamic Category Management

Categories are not fixed. The `CategoryBase` allows new categories to be created dynamically when new items are added.

This makes the system easier to extend without structural changes.

7.10 R7 - Home Delivery

The system supports home delivery through the DeliveryHub component. Customers can request a delivery during checkout. Delivery requests are first queued and later scheduled when the payment is completed.

Once the payment is successful, the system triggers the delivery process and finalises the order dispatch.

7.11 R8 - Delivery Pricing

Delivery pricing is computed dynamically based on both the total weight of the order and the delivery distance. The system applies different rules depending on the customer's discount plan.

Heavier orders and longer distances increase the delivery cost, while premium plans may reduce or eliminate the fee.

8 Testing Strategy

The system has been tested using JUnit 5 unit tests covering the main business logic, command validation, and category management.

Recent updates added more detailed tests for the category system, including: - prevention of duplicate category creation, - validation of discount limits, - handling of invalid category lookups.

These tests ensure that the system correctly enforces data consistency in the inventory and category modules.

8.1 Testing Approach

The testing strategy is based on unit testing rather than integration or system testing. Each test class focuses on a specific part of the system and checks observable behaviour instead of implementation details.

This approach ensures that internal refactoring does not break external behaviour.

Exception handling is also tested to make sure invalid inputs are properly detected and handled.

8.2 Test Coverage by Component

The tests are organized by functional domain:

- **User Management**: authentication, registration, and session handling are tested in `UserBaseAndSessionTest`.
- **Inventory System**: stock management, restocking, destocking, and error cases are tested in `InventoryTest`.
- **Customer Management**: customer registration, retrieval, and discount plan subscription are tested in `CustomerBaseTest`.
- **Customer Domain Model**: object consistency, equality, and discount plan behavior are tested in `CustomerTest`.

9 Workload Distribution

The workload was shared between both team members, with a natural split between system implementation and documentation-oriented tasks. Both members participated in the initial design discussions in order to ensure a consistent global architecture before implementation.

9.1 Task Contribution Table

Member	Design	Code	UML	JUnit	Tasks / Classes
Moncef	High	High	System architecture	Core tests	CLI, Inventory, Users, Sessions, Discount system
Raïcha	Medium	Medium	UML formatting	Test docs	Report writing, UML presentation, requirements mapping, test scenarios

10 Conclusion

This project allowed us to design and implement a complete supermarket checkout system using an object-oriented approach. The final architecture is modular, with clear separation between the main components of the system.

The use of design patterns such as Strategy helped make the pricing and discount logic more flexible, while repository-style classes improved data organisation and consistency.

The system is also structured in a way that makes future extensions easier, especially for adding new features or rules.

Overall, the project demonstrates a clean and maintainable design that meets the main functional requirements while remaining simple to extend.